

Experiment 2: Spam Email Classification using Naive Bayes and K-Nearest Neighbors

1 Aim and Objective

The aim of this experiment is to classify emails as spam or non-spam using the Spambase dataset. The objectives are to:

- Perform exploratory data analysis (EDA) on the spam email dataset.
- Train and compare three variants of Naive Bayes classifiers – Gaussian, Multinomial, and Bernoulli.
- Train a K-Nearest Neighbors (KNN) classifier and tune its hyperparameters using both Grid Search and Randomized Search with 5-fold cross-validation.
- Compare KNN's KDTree and BallTree implementations for efficiency.
- Evaluate all models using standard classification metrics and analyze their computational (time) complexity, both theoretically and experimentally.

2 Theory

Naive Bayes classifiers are probabilistic models based on Bayes' theorem with a "naive" assumption of conditional independence between features given the class label. Three variants were used:

- **Gaussian NB** assumes continuous features are normally distributed within each class.
- **Multinomial NB** models feature counts/frequencies and is traditionally used for text/word-frequency data.
- **Bernoulli NB** assumes binary/boolean features, treating each feature as present or absent.

K-Nearest Neighbors (KNN) is a non-parametric, instance-based classifier that assigns a class label based on the majority vote of the k nearest training samples, using a distance metric (e.g. Euclidean or Manhattan). Since KNN relies on distance computation, feature scaling is critical. Two tree-based data structures, **KDTree** and **BallTree**, are commonly used to speed up nearest-neighbor search compared to the brute-force approach, especially for higher-dimensional data.

3 Dataset

The dataset used is the **Spambase** dataset (`spambase_csv.csv`), containing 4,601 emails and 58 columns:

- 48 `word_freq_*` features – percentage of words in the email matching a specific word.
- 6 `char_freq_*` features – percentage of characters matching specific symbols (e.g. `;`, `(`, `[`, `!`, `$`, `#`).
- 3 `capital_run_length_*` features – statistics on runs of capital letters (average, longest, total).
- `class` – the binary target (1 = spam, 0 = not spam).

The dataset has no missing values. The target class is moderately imbalanced, with 39.4% of emails labeled as spam. Most word/character frequency features are heavily right-skewed (skewness often exceeding 5–30), since most emails contain a given word or symbol zero or very few times.

4 Exploratory Data Analysis (EDA)

EDA was performed to understand feature distributions and detect data quality issues. Figure 1 presents the EDA panel generated for the dataset. Key observations:

- All 57 predictor columns are numeric; no categorical encoding was required.
- Several features (e.g. `word_freq_3d`, `char_freq_%23`, `word_freq_table`) have extreme skewness and kurtosis, indicating rare but large outlier values (e.g. `capital_run_length_longest` has a max of 9,989 vs. a median of 15).
- No missing values were found in any column.
- The target class distribution (60.6% non-spam vs. 39.4% spam) is moderately imbalanced but not severe enough to require resampling.

5 Code / Implementation Details

The workflow was implemented using `scikit-learn`. Key steps:

1. **Train/test split:** An 80/20 split was applied (`random_state=42`), with `class` as the target and all other 57 columns as features.
2. **Naive Bayes models:** Gaussian NB, Multinomial NB, and Bernoulli NB were each trained on the (unscaled) raw feature values.
3. **Feature scaling:** A `StandardScaler` was fit on the training set and applied to both training and test sets, used specifically for the distance-based KNN models.
4. **KNN baseline:** A default `KNeighborsClassifier` ($k = 5$) was trained as a baseline.
5. **KDTree vs. BallTree vs. Brute:** The three KNN search algorithms were timed for training and prediction to compare their computational efficiency.

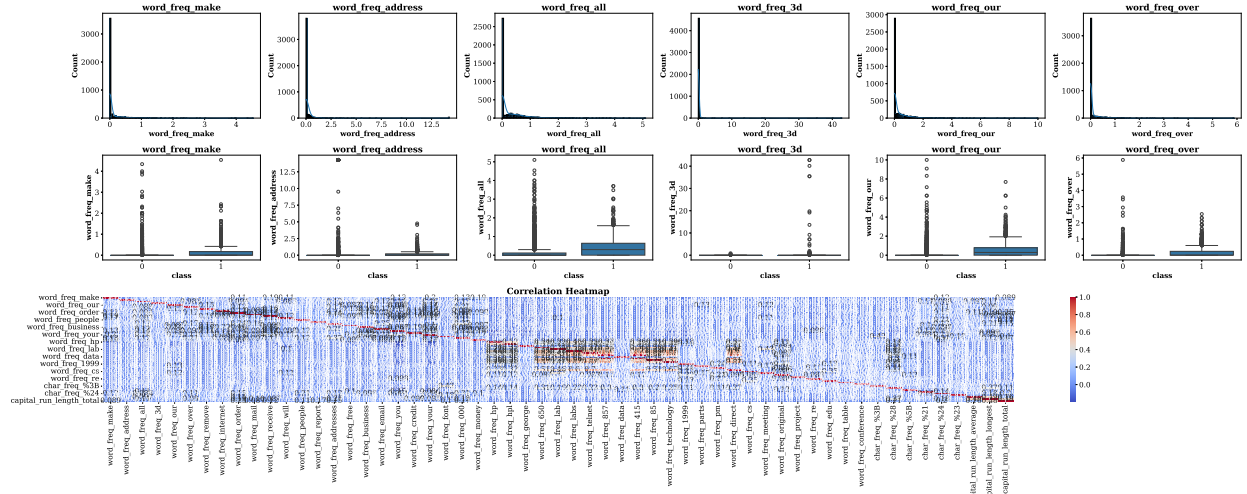


Figure 1: Exploratory data analysis panel for the Spambase dataset, including target class distribution, feature distributions/skewness, and correlations among key features.

Representative code for the Naive Bayes baseline:

```
mnb = MultinomialNB()
mnb.fit(x_train, y_train)
y_pred_mnb = mnb.predict(x_test)
```

6 Hyperparameter Tuning

KNN hyperparameters were tuned using two strategies, both with 5-fold cross-validation and accuracy scoring:

- **GridSearchCV** over $n_neighbors \in \{3, 5, 7, 9, 11, 15\}$, $weights \in \{\text{uniform, distance}\}$, $metric \in \{\text{euclidean, manhattan}\}$.
- **RandomizedSearchCV** over the same grid (plus `minkowski` as a metric option), sampling 20 combinations.

Both methods converged on the same optimal configuration: $n_neighbors = 11$, **weights = distance**, **metric = manhattan**, with a best cross-validation accuracy of **0.9255**. GridSearchCV took 0.608s to complete, while RandomizedSearchCV took 0.494s – faster since it evaluates a random subset of the combinations rather than the full grid.

```
param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11, 15],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}
grid_search = GridSearchCV(KNeighborsClassifier(), param_grid,
                           cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(x_train_scaled, y_train)
```

7 Comparison Tables

Table 1: Naive Bayes model comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8208	0.7861	0.8806
Precision	0.7193	0.7644	0.9070
Recall	0.9462	0.7154	0.8000
F1	0.8173	0.7391	0.8501
ROC-AUC	0.9419	0.8639	0.9555

Table 2: KNN accuracy across values of k (unscaled, default settings)

k	Accuracy	Precision	Recall	F1
1	0.8936	0.8744	0.8744	0.8744
3	0.8914	0.8836	0.8564	0.8698
5	0.8936	0.8989	0.8436	0.8704
7	0.8947	0.9127	0.8308	0.8698
9	0.8903	0.9048	0.8282	0.8648
11	0.8969	0.9178	0.8308	0.8721

Table 3: GridSearchCV vs. RandomizedSearchCV

	GridSearchCV	RandomizedSearchCV
Best k	11	11
Metric	manhattan	manhattan
Weights	distance	distance
CV Accuracy	0.9255	0.9255
Execution Time (s)	0.608	0.494

Table 4: KDTree vs. BallTree ($k = 5$, scaled features)

	Accuracy	Training Time (s)	Prediction Time (s)
KDTree	0.8936	0.01164	0.10731
BallTree	0.8936	0.00489	0.09879

8 Complexity Analysis

The brute-force KNN search algorithm was fastest overall (0.0077s) on this dataset, ahead of BallTree (0.0927s) and KDTree (0.1130s). This is consistent with the theoretical expectation that tree-based structures offer the largest benefit at higher dimensionality or larger sample sizes; with

Table 5: 5-fold cross-validation: best Naive Bayes (Bernoulli) vs. best KNN

	Naive Bayes (Bernoulli)	Best KNN
Fold 1	0.9117	0.9293
Fold 2	0.8804	0.9130
Fold 3	0.9076	0.9280
Fold 4	0.9008	0.9321
Fold 5	0.9035	0.9253
Average	0.9008	0.9255

Table 6: Theoretical time complexity (n = samples, d = features)

Model	Training	Prediction
Naive Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(nd \log n)$	$O(\log n)$ avg
BallTree	$O(nd \log n)$	$O(\log n)$ avg

Table 7: Experimental time analysis

Model	Training (s)	Prediction (s)
Gaussian NB	0.003244	0.000930
Multinomial NB	0.001834	0.000648
Bernoulli NB	0.002804	0.001012
Best KNN	0.001100	0.026953

57 features and $\sim 4,600$ samples, the overhead of tree construction outweighs its search-time savings. Naive Bayes models train and predict substantially faster than KNN overall, reflecting their $O(nd)/O(d)$ complexity versus KNN’s comparatively expensive prediction step.

9 Plots

Figure 2 shows three evaluation panels for the best-tuned KNN model: the confusion matrix, the spread of 5-fold cross-validation accuracy, and a comparison of training vs. prediction time.

10 Results

- Among the Naive Bayes variants, **Bernoulli NB** performed best (accuracy 0.8806, F1 0.8501), consistent with the binary ”word present/absent” nature of many spam indicators. Gaussian NB had the highest recall (0.9462), making it more aggressive at catching spam at the cost of precision, while Multinomial NB performed weakest overall.
- The tuned **KNN** model ($k = 11$, weights = distance, metric = manhattan) substantially outperformed both the untuned KNN baseline (0.79 accuracy) and all Naive Bayes variants, reaching **0.9153** test accuracy and **0.9255** mean cross-validation accuracy.

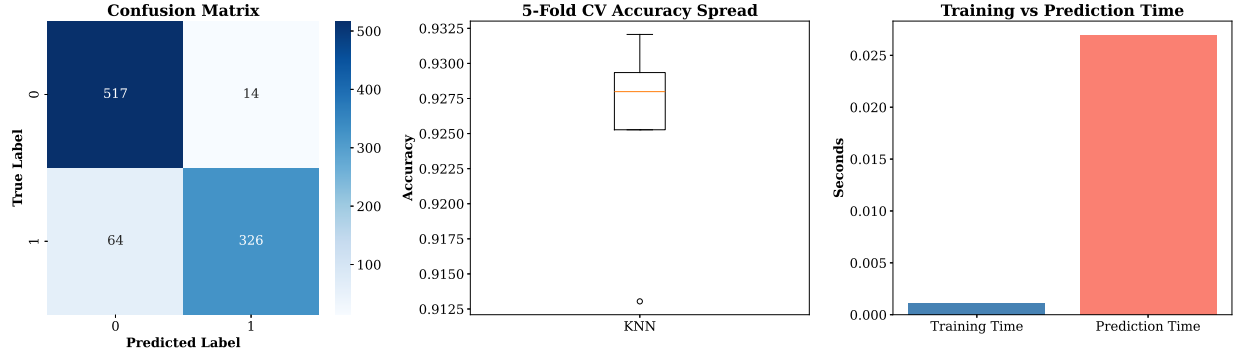


Figure 2: KNN evaluation panel: (left) confusion matrix, (middle) 5-fold cross-validation accuracy spread, (right) training vs. prediction time.

- Both GridSearchCV and RandomizedSearchCV found the identical optimal hyperparameters, with RandomizedSearchCV completing modestly faster.
- KDTree and BallTree produced identical accuracy to brute-force KNN (as expected, since they compute exact nearest neighbors), differing only in computation time.

11 Conclusion

Tuned KNN was the strongest overall classifier on this dataset, achieving 92.55% mean cross-validation accuracy compared to at most 88.06% for the best Naive Bayes variant (Bernoulli). Feature scaling was essential for KNN, since it relies directly on distance computations across features with very different scales. However, this performance gain came with a computational cost: KNN’s prediction step is markedly slower than any Naive Bayes variant, since it requires distance computations against training samples at inference time rather than the lightweight probability lookups used by Naive Bayes. For this dataset’s size, brute-force search was more efficient than the KDTree/BallTree alternatives, showing that the benefit of spatial indexing structures depends heavily on dataset dimensionality and size. Overall, the experiment highlights a practical accuracy-vs-speed trade-off: Naive Bayes offers fast, lightweight classification with solid but lower accuracy, while tuned KNN offers superior classification accuracy at a higher prediction-time cost.